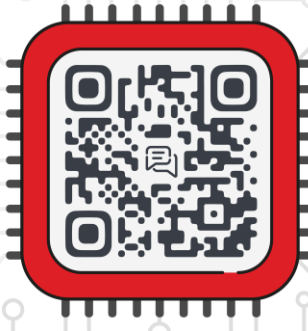




2



23



2760 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Revision and More

د. هبة حاتم



RB Informatics; 16/04/2025

اللغات الصورية

مراجعة للمحاضرة السابقة

:Theory of computing

تعرف بالعموم على أنها علم يدرس ثلاثة حقول:



:Automata Theory ببساطة هي علم يحاول أن يجد نماذج رياضية للآلات، أي Mathematical Models لل Machines تقوم بحل مسألة معينة.

:Computability Theory هل هذه المسألة قابلة للحل أم لا؟ (سيتم دراستها لاحقاً).

:Complexity Theory تدرس هل هذه المسألة هي Hard or Easy؟ وإذا كانت صعبة فما هي درجة تعقيدها*

* درجة التعقيد هنا تختلف عنها في الخوارزميات.

مرجع المادة: كتاب Michel Spiser مايكل سبايزر



🔗 قدم العالم "Chomsky" (عالم شهير جداً في اللغويات واللسانيات) أربع تصنيفات للغات المحلية، وذلك بحسب درجة صعوبتها وبحسب القواعد التي تحكمها.

🔗 تتراوح التصنيفات الأربعة من الأعداد 0 Type إلى الأقل تعقيداً 3 Type. وسنقوم بدراسة الدرجات الأربع من تصنيفات اللغات، ولكن هذه اللغات تختلف عن لغات البرمجة وهي عبارة عن مجموعة قد تكون منتهية وقد تكون غير منتهية من ال Strings أي "Set of Symbols"، وبالتالي فإن أي آلة قادرة على أن تدرك لغة هي آلة

🔗 استلهم علماء الحواسيب من العالم Chomsky بحسب درجة التعقيد هذه الهرمية، وقامو بتعميمها على اللغات التي يمكن أن تدركها الآلات باختلاف التعقيد، فالآلة الأقل تعقيداً هي "Finite State Machine" أو الآلة ذات الحالة المنتهية "FSM" وبدورها مقسمة لمجموعة أقسام ستتم دراستها لاحقاً.

1. Regular Languages

هي اللغات التي تدركها ال Finite State Machine والقواعد هي Regular Rules

2. Context Free Languages (اللغات الحرة من السياق)

أما الأوتومات التي تدرك مثل هذا النوع من اللغات هي Pushdown Automaton وسندرسها لاحقاً بعد 3-4 محاضرات

3. Context Sensitive Languages (اللغات الحساسة للسياق)

والأوتومات التي تدرك مثل هذا النوع من اللغات هي Linear Bounded Automaton

■ مثلاً، لدينا المتغير X موجود مرتين:

(1) مرة Global (على مستوى البرنامج بالكامل) وقيمه 10

(2) مرة محلي (داخل Function) قيمته 5

■ عند كتابة X لا يكفي فقط رؤية X وتفسيره، بل علينا معرفة أين نحن الآن، داخل التابع أم خارجه؟

داخل التابع: $X = 5$ خارج التابع: $X = 10$

وبالتالي فإن معنى X يعتمد على السياق الذي ظهر فيه لكي نعرف أي نسخة منه سنستخدم

4. Turing Machine (الآلة التجريدية)

عند هذه اللغة القواعد غير محكومة بشيء، وهي العملية الأشمل لكل مما سبق، وهي اللغة القادرة على حل أصعب المسائل حتى مسألة التوقف "Halting Problem" نفسها

ما هي مسألة التوقف Halting Problem؟

إذا أردت كتابة Automata Test، ستحتاج إلى معرفة فيما إذا كان برنامج معين Valid بلغة Java (مثلا الكود مكتوب بشكل صحيح، القواعد النحوية صحيحة)؟

نعم هذا ممكن، لأن التحقق من الشكل أو الصيغة "Syntax" شيء آلي تستطيع حله بـ DFA أو تقنيات تحليل القواعد، أما إذا طلب منا كتابة Automata Testing يقرر: هل برنامج معين عند تشغيله سينتهي، أم أنه سيبقى في حلقة لانهائية (دون تشغيله فعلياً وتجربة جميع احتمالاته).

هذا مستحيل! لماذا؟

لأنك لا تستطيع أن تعرف بشكل أكيد إذا كان البرنامج سيتوقف أم لا دون تنفيذه فعلياً أو متابعة عملياته، ومن الممكن أن يعتمد البرنامج على مدخلات أو يحسب أشياء معقدة جداً، فمن المستحيل معرفة إذا ما كان سيتوقف في جميع الحالات. هذه هي Halting Problem: هل يوجد خوارزمية / آلة تستطيع أن تقرر (لكل برنامج ولكل مدخل) إذا كان البرنامج سيتوقف أم لا؟ والجواب جاء بأنها **مسألة لا يمكن حلها**. سنبحث كيف ستستطيع Turing Machine حل هذه المسألة.

تختلف أنواع الآلات في تصنيف تشومسكي من حيث طبيعة قواعد الانتقال والقيود على استخدام الذاكرة.

تصنيفات الـ Automata من الأضعف للأقوى بحسب الذاكرة والقوة الحسابية

1. Finite State Machine (FSM):

- ❖ Memoryless تقريباً بدون ذاكرة، أو نعتبر ذاكرتها محدودة بعدد الحالات
- ❖ تتعامل مع لغات منتظمة (Regular Languages)، مثال: آلة تحقق إذا كلمة فيها عدد زوجي من 0

2. Pushdown Automaton (PDA):

- ❖ الـ Memory الخاصة بها تستخدم Stack
- ❖ تتعامل مع اللغات الخالية من السياق Context Free Languages، عملياً هذه اللغات هي Proportional للدخل، أي حسب الدخل تتحدد الـ Memory، مثال: التحقق من توازن الأقواس $\{(())\}$ أو $\{[()]\}$

3. Linear Bounded Automaton (LBA):

- ❖ الذاكرة: عندها شريط (Tape) مثل Turing Machine، لكن طوله محصور بحجم الإدخال (محدود خطياً)
- ❖ تتعامل مع اللغات الحساسة للسياق (Context Sensitive Languages)، مثال: مطابقة عدد معين من الرموز حسب القواعد المعقدة مثل $\{a^n b^n c^n \mid n \geq 1\}$

4. Unlimited Memory / Turing Machine (TM):

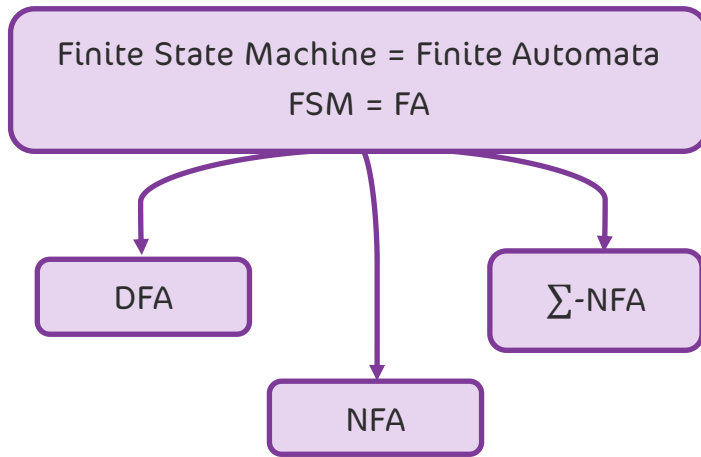
❖ الذاكرة: عندها شريط غير محدود ← ذاكرة غير محدودة

❖ تتعامل مع كل شيء يمكن حسابه (Recursively Enumerable Languages)، مثال: تنفيذ برامج معقدة مثل برامج بايثون أو أي خوارزمية

ملاحظة هامة:

- كل آلة أقوى من الآلة التي قبلها
- كلما زادت الذاكرة زادت القدرة على التعامل مع لغات وتعقيدات أكبر
- كل فئة تحتوي على الفئات الأضعف منها (TM تحتوي LBA و PDA و FSM)

تذكر:



Finite Automata سندرس فيها:

- 1- Deterministic Finite Automata (DFA)
- 2- Non-Deterministic Finite Automata (NFA)
- 3- Σ -None-Deterministic Finite Automata (Σ -NFA)

وسندرس كيفية التحويل من NFA إلى DFA وكيف سنصغر الأوتومات إلى درجة معينة لاحقاً

- تذكر أن المثال 2 من المحاضرة السابقة (مثال الـ Heatwave) هو مثال عن الـ FSM ولكنه أيضاً نوع خاص من الـ FSM اسمه DFA.
 - لماذا؟
- لأنه عندما يأتي لكل حالة Input Symbol يكون لها Transition (انتقال) وحيد إلى حالة واحدة ومحددة، الموضوع حتمي لهذا السبب هو DFA (Deterministic)

مثال: صمم Automata تدرك اللغة B التالية:

$Let B = \{ w \mid w \text{ has an even number of 1s} \}$

مناقشة قبل الحل:

هل هذه اللغة منتهية أم لا؟
لا، ليست منتهية لأننا نستطيع كتابة عدد لانهائي من الـ Strings مؤلفة من الـ Symbols (0, 1) والتي تحوي عدد زوجي من الـ 1s، "0 and 1 are symbols of Σ "

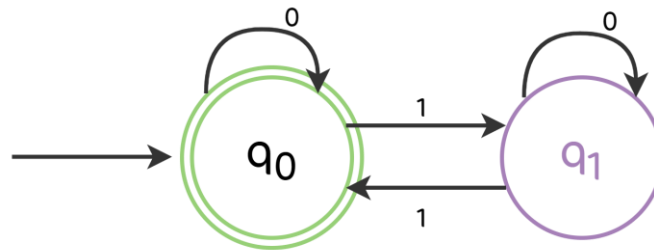
- تذكرة: أي لغة تدركها الـ FSM هي Regular Language.
- انتبه: لعدد الحالات التي سنضعها بالـ Automata التي نريد تصميمها، لا يتم انتقاء الحالات بشكل اعتباطي وإنما نضع لكل حالة هدف.

في هذا المثال لدينا حالتان: (1 even number of 1s (2 odd number of 1s

ماهي الحالة الابتدائية؟ even؟ odd؟

في الحالة الابتدائية لا يوجد شيء لا صفر ولا حتى واحد، أي لدينا عدد صفر من الواحدات، والصفر عدد زوجي

Start State is an accept (q_0)



شرح:

q_0 اذا حصلت على دخل "1" ← عدد الواحدات 1 فردي ← تؤدي إلى حالة أخرى هي q_1 ← q_1 تمثل عدد الواحدات الفردي

q_1 اذا حصلت على دخل "1" ← عدد الواحدات 2 زوجي ← تؤدي إلى q_0

q_1 اذا حصلت على دخل "0" ← عدد الواحدات 1 فردي ← تبقى بـ q_1

q_0 اذا حصلت على دخل "0" ← عدد الواحدات يبقى زوجي ← تبقى بـ q_0

11	→	زوجي q_0	✓
111	→	فردى q_1	✗
110	→	زوجي q_0	✓

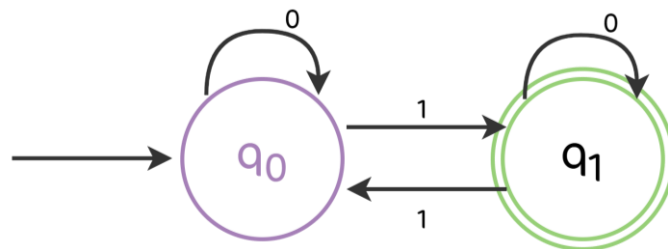
مثال: صمم Automata تدرك اللغة C التالية:

Let $C = \{w \mid w \text{ has an odd number of 1s}\}$

الفرق الوحيد بين هذا الـ Automaton والذي قبله هو الـ Accept State

انتبه: ال Accept State تحدد لنا عمل ال Automaton وبالتالي إذا أخطأنا فيها فسنحصل على أغلاط بتصميم ال Automaton

وفقاً لنص المسألة الحالة الابتدائية ليست Accept State لأن الحالة الابتدائية تحوي عدد زوجي من الواحدات



للتحقق فيما إذا كانت الـ Automaton صحيحة لأَمْ لا، نقوم باختبارها. باختبار الـ String على الـ Automaton وليس اختبار عقلي / شفهي

111	→	q ₁	✓
11	→	q ₀	✗
10	→	q ₁	✓

ما سنأخذه بعد قليل هو مقدمة عن درس كامل اسمه **Regular Expressions**



Regular Expressions

❖ لتعريف الـ Regular Expressions سنبدأ بتعريف الـ Regular Operations

❖ بفرض لدي (A, B) Two Regular Languages:

1. Union: $A \cup B = \{ w \mid w \in A \text{ or } w \in B \}$ $A \cup B$ عبارة عن String بحيث هذا الـ String إما ينتمي لـ A أو ينتمي لـ B $\alpha; \quad A = \{ \text{Orange}, \text{Banana} \}$ $B = \{ \text{Apple}, \text{Tomato} \}$ $\Rightarrow A \cup B = \{ \text{Orange}, \text{Banana}, \text{Apple}, \text{Tomato} \}$ $A \cup B = B \cup A$

الاجتماع عملية تبديلية حيث:

2. Concatenation: $A \circ B = AB = \{ xy \mid x \in A \text{ and } y \in B \}$

وتختلف عنها في الـ C++

 $A \circ B \neq B \circ A$

عملية غير تبديلية

Ex: $A = \{ \text{hi}, \text{hello} \}, \quad B = \{ \text{stu}, \text{Tea} \}$ $A \circ B = \{ \text{hi}, \text{stu}, \text{hello stu}, \text{hi Tea}, \text{hello Tea} \}$ $\neq$ $B \circ A = \{ \text{stu hi}, \text{stu hello}, \text{Tea hi}, \text{Tea hello} \}$ 3. Star: $A^* = \{ x_1 \dots x_k \mid \text{each } x_i \in A \text{ for } k \geq 0 \}; \epsilon \in A^* \text{ always}$ A^* (بدون دائرة): تعطينا مجموعة، وهذه المجموعة غير منتهية من كل المكونات التي يمكن أن تنتج من هذه اللغة

1- الـ Star: لا تطبق على لغتين وإنما على لغة واحدة فقط

2- تعطينا كل الاحتمالات غير المنتهية، بالإضافة إلى الـ Empty String، أي الـ ϵ ، حيث $\epsilon \in A^*$ وهي تميز الـ A^* بالتالي الـ Star هي مجموعة غير منتهية تحوي ابسيلون ϵ Ex: Let $A = \{ \text{good}, \text{bad} \}, B = \{ \text{boy}, \text{girl} \}$ $A \cup B = \{ \text{good}, \text{bad}, \text{boy}, \text{girl} \}$ $A \circ B = \{ \text{good boy}, \text{good girl}, \text{bad boy}, \text{bad girl} \}$ $A^* = \{ \epsilon, \text{good}, \text{bad}, \text{good good}, \text{bad bad}, \text{good bad}, \text{bad good}, \text{bad bad}, \text{good good good}, \text{good good bad}, \dots \}$ Regular
Operations

Regular Expressions

1. تبني من:

 Σ (أبجدية اللغة)، Σ members، ϕ ، ϵ ايسيلون

2. ولكن كيف؟

باستخدام الـ Regular Operations ($\cup$, * , o) التي تتشكل منها

Examples For Regular Expressions

1. $(0 \cup 1)^* = \Sigma^*$ gives all strings over Σ

كل الاحتمالات المتعلقة بـ 0، 1

2. $\Sigma^* 1$ gives all strings that end with 1 Σ^* تعطينا كل الاحتمالات المتعلقة بـ 1، 0 أما $\Sigma^* 1$ كل الاحتمالات المنتهية بـ 1

← Regular Expression لهذه المرحلة نستخدمها لتوصف Language تدرك من قبل Finite Automata

3. $\Sigma^* 11 \Sigma^* = \text{all strings that contain } 11 = L(M_1)$

توصف آلة تحوي على واحدتين (11) متتاليتين

الهدف الأساسي هو: إظهار Finite Automation يكافئ إلى Regular Expressions

(أي سنتعلم التحويل من FA إلى Regex ومن Regex إلى FA) حيث Regex = Regular Expression

DFA: Deterministic Finite Automata

عبارة عن نوع من أنواع الـ Automata ذو حالة حتماً معروفة، بمعنى: كل State يدخل عليها Symbol ستنقل إلى State ثانية وواحدة فقط، لا يمكنها الانتقال من أجل Input Symbol إلى أكثر من State

Definition of DFA

1. Q : a finite set of states (مجموعة ذات عدد منته من الحالات)2. Σ : a finite set of input symbols (alphabet)

↪ قد تكون أبجدية اللغة (0, 1) أو (A, B, C)

↪ الأبجدية الإنكليزية 26 حرف (A → Z)

↪ الأبجدية العربية 28 حرف (أ ← ي)

3. q_0 : a start state (وهي حالة ابتدائية واحدة فقط)

4. F : set of finite / accepting states

5. δ : a transition function, which is a mapping between $Q \times \Sigma \rightarrow Q$

بال DFA لدينا من أجل كل Symbol حالة انتقال واحدة فقط، إذا حصلت على 0 ستذهب إلى مكان، إذا حصلت على 1 ستذهب إلى مكان، لا تذهب إلى مكانين

$\Rightarrow$ A DFA is defined by the 5 – tuple: $\{ Q, \Sigma, q_0, F, \delta \}$

Key Characteristics of DFAs

1. Determinism (الاحتمية):

لكل State (q) عند دخل 0, 1, whatever عملياً لدينا Transition واحد فقط، لا غير

No Ambiguity: لا غموض، كيف يأتي الغموض؟

الغموض يحصل عند حالة معينة ومع قراءة رمز معين، يكون هناك أكثر من خيار للانتقال (كما في الـ NFA)

مثال

$$q_1 - 0 \rightarrow q_2$$

$$q_1 - 0 \rightarrow q_1$$

هنا لا نعرف لأي حالة نذهب، هذا غموض في الـ DFA غير مسموح

2. Finite States and Alphabet:

لديها عدد منته من الحالات والـ Σ لها عدد منته من الرموز

3. Input Processing:

كيف يحصل؟ نبدأ بمعالجة السلسلة (الـ Strings) ابتداءً من الحالة الابتدائية من اليسار إلى اليمين واحداً تلو الآخر، نبدأ بأول String من اليسار إلى اليمين ثم أول String يدخل، ونلاحظ الـ String الثاني وهكذا نعالج هذه السلسلة للوصول للحالة. إذا الحالة التي وصلنا لها هي جزء من الـ Finite State

$\Rightarrow$ Machine Automaton accepted this string, otherwise it rejected it

4. No Σ – Transition:

أما حالة أن تكون الـ NFA لا تتساوى تماماً مع الطاقة الحسابية للـ NFA، أي أن الـ NFA لا تقدم شيئاً إضافياً عما تقدمه الـ DFA فهذه الحالة موجودة بالـ NFA

5. Computer Power:

الـ DFA تتساوى تماماً مع الطاقة الحسابية للـ NFA، أي أن الـ NFA لا تقدم شيئاً إضافياً عما تقدمه الـ DFA
كلتاهما Memoryless تقريباً

ماذا يفعل الـ DFA عندما يقرأ An Input String؟

❖ Input: a word w is in Σ^*

❖ الـ w تكون Acceptable من قبل الـ Automaton / Machine

❖ Steps:

أولاً:

نبدأ من الـ q_0 ، من أجل كل Input Symbol ندرس الحالة التالية لها، وهكذا نكرر العملية حتى تفنى الـ Symbols.

DFA accepted the w



إذا انتهينا بحالة قبول (One of Accepting States)

Otherwise:

DFA rejected the w

ماهي اللغة التي تدركها الـ DFA؟

الجواب: Regular Language

أين موقع الـ DFA / Regular Language بالـ Chomsky Hierachary (هرمية)

تشومسكي؟

الجواب: موقع الـ DFA / Regular Language عملياً بأدنى درجة من درجات الهرم والتي تعتبر أقل درجة تعقيد، وبالتالي عندما أريد التعبير عن اللغة التي تدركها الـ Machine أقول:

$L(A)$ be a language recognized by a DFA A

$\Rightarrow L(A)$ is called a "Regular Language"

وموقعها بأدنى درجات التعقيد لهرمية تشومسكي

خطوات بناء الـ DFA

1. تحديد أبجدية اللغة: Σ

2. نقرر ما هي حالاتها: Q

3. نقرر ماهي الحالة الابتدائية والحالة النهائية (الحالات النهائية)

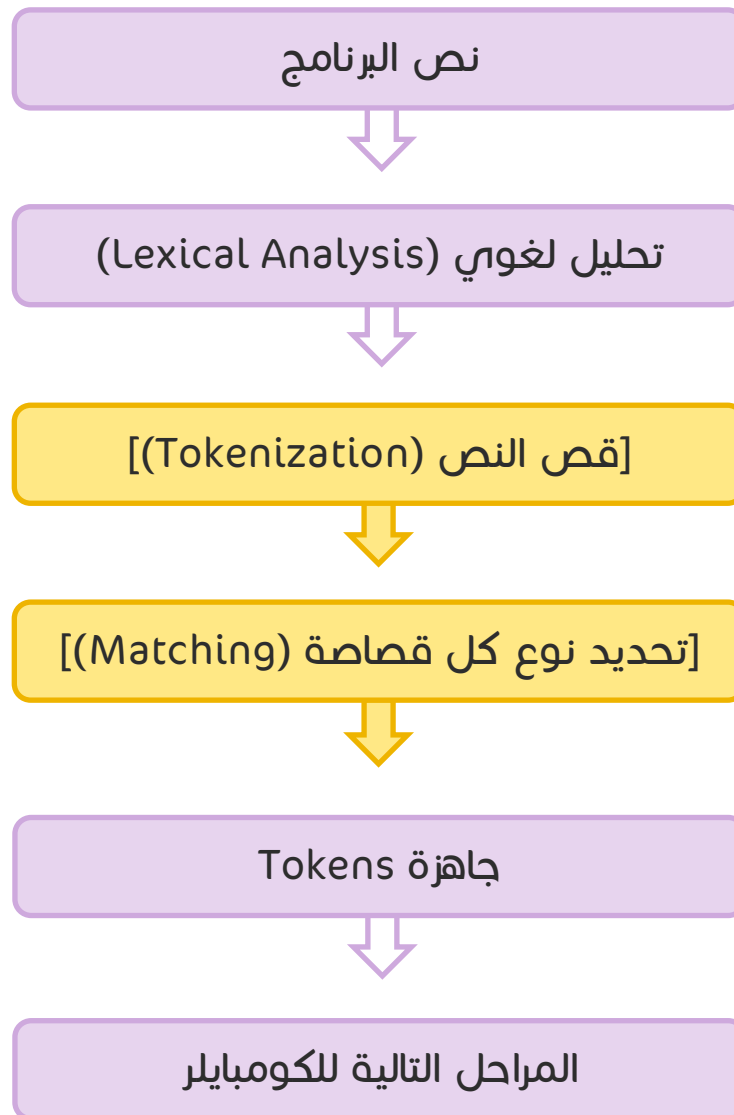
4. كتابة الـ "Transition Function": δ

الـ Finite State: عبارة عن مجموعة من الـ accepting states، وكل state ليست ضمن مجموعة الـ accepting states هي عبارة عن Non-accepting state، وبالتالي إذا انتهت بوحدة من هذه الحالات هذا يعني أن الـ Automaton رفض الـ w / word

Applications of DFAs

1- Lexical Analysis (التحليل المعجمي)

1. المبرمج كتب قصة (كود)
2. الكومبايلر يجب أن يقرأ القصة كلمةً كلمةً (وليس حرفاً حرفاً)
3. لذلك أول ما سيقوم به هو Lexical Analysis
4. أثناء التحليل، يبدأ بعمل Tokenization: يقطع القصة إلى كلمات وجمل صغيرة
5. أثناء التقطيع يحاول القيام بـ Matching: يقرر نوع كل كلمة
6. النتيجة النهائية: القصة تحولت إلى مجموعة بطاقات (Tokens) مرتبة ومنظمة لكي يفهمها أكثر في المراحل التالية

توضيح بالرسم:

ما هو ال Compiler؟

ال Compiler هو برنامج مسؤول عن:

- ❖ أخذ كود البرنامج (مثل java, C++, C#,....)
- ❖ فهمه وتحويله إلى لغة يستطيع الحاسوب فهمها (Machine Code)

من مراحل عمل الكومبايلر الأساسية: Lexical Analysis

ما هو ال Lexical Analysis؟

ال Lexical Analysis هو أول مرحلة في عمل الكومبايلر، مهمته الأساسية:

- ❖ قراءة الكود المكتوب ك String
- ❖ تقسيمه إلى وحدات صغيرة مقسومة اسمها Tokens

لماذا نحتاج Lexical Analysis؟

- ❖ لأن الكود الخام عبارة عن حروف مبعثرة، والكومبايلر يحتاج فهمها كأجزاء منظمة
- ❖ لتسهيل المرحلة التالية (Syntax analysis)

ما هو ال Token؟

ال Token هو عبارة عن وحدة أساسية في الكود البرمجي، أمثلة عنه:

أمثلة عنه	Token ال
If while return	Keyword
X sum	Identifier
= * - +	Operator
3.14 100 5	Number
{ } , ;	Punctuation

الكومبايلر لا يفهم النص حرفاً حرفاً بل يفهمه ك مجموعة Tokens



ما هو الـ Tokenization؟

الـ Tokenization ببساطة هو:

- ❖ عملية تقطيع الكود إلى أجزاء صغيرة (Tokens)
- ❖ تحديد بداية ونهاية كل جزء

مثلاً `int x = 5;`

عملية الـ Tokenization تنتج:

int	→	Keyword
x	→	Identifier
=	→	Operator
5	→	Number
;	→	Punctuation

ما هو الـ Matching؟

- ❖ الـ Matching يحدث داخل عملية الـ Tokenization
- ❖ عندما يقرأ الكومبايلر الكود حرفاً حرفاً، يحاول مطابقة الحروف مع أنماط محددة (Patterns) مثلاً إذا قرأ حروف مثل `int` مع بعضها فسيطابقها مع نمط الأعداد
- ❖ يعتمد على Regex Engines (محركات Regex)

كيف يتم تفكيك الـ String إلى Substrings؟

الخطوات التي يقوم بها الكومبايلر لتفكيك النص:

- (1) يبدأ بقراءة الأحرف واحداً واحداً
- (2) يحاول تحديد إذا كانت الأحرف الحالية تعبر عن (جزء من كلمة / رقم / رمز)
- (3) عندما يصل إلى علامة نهاية (مثل فراغ / فاصلة / علامة مساواة) يفهم أن الـ Token انتهى
- (4) يأخذ الجزء الذي يقرأه كـ Substring ويصنع منه Token

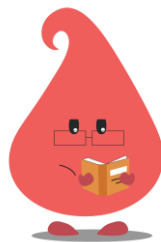
خلاصة ما تعلمناه

المفهوم	شرح مبسط
Compiler	برنامج يحول الكود إلى لغة الآلة
Lexical Analysis	أول خطوة: تقطيع الكود إلى Token
Tokenization	عملية تقطيع النص إلى Tokens
Matching	مقارنة النصوص بأنماط لاستخراج نوع ال Token
Substrings	تفكيك النص الخام (String) إلى أجزاء صغيرة باستخدام قراءة متتابعة ذكية

- يعتمد ال Compiler أثناء ال Lexical Analysis بشكل رئيسي على ال DFA، حيث يستخدم DFA في تحديد بداية ونهاية كل Token عن طريق ال Matching
- ال Tokenization يتم على مستوى الكود لكن ال Substring يتم على مستوى القيم النصية أثناء التنفيذ، وبالتالي فإن:
- Lexical Analysis: هي الخطوة الأولى في التحقق من هذا ال Syntax الذي نكتبه فيما إذا كان صحيحاً أم لا (الخطوة الأولى من خطوات الترجمة) وبالتالي هذا يعتمد على ال DFA

2- Pattern Matching

Pattern يعني نمط، ال DFA يُستخدم في ال Pattern Matching من أجل ال Valid Pattern مع Pattern آخر وهذا يحصل في Regex Engines، ولاحقاً سنتعلم كتابة Regular Expressions. أي أن ال Regex يقدم Valid Phone Number, Valid E-mail,....، فال DFA يستخدم من أجل هذه المطابقة.



3- Hardware Design

أيضاً يُستخدم الـ DFA في الـ Hardware Design بالمتحكمات ذات الحالة النهائية داخل الدارات الكهربائية Finite State Controllers in Circuits

EX:

Build a DFA for the following language:

$$L = \{ w \mid w \text{ is a binary string that contains } 01 \text{ as a substring} \}$$

00 ❌

1101 ✅

لا يهمنا مكان تموضع الثنائية "01" داخل الـ String

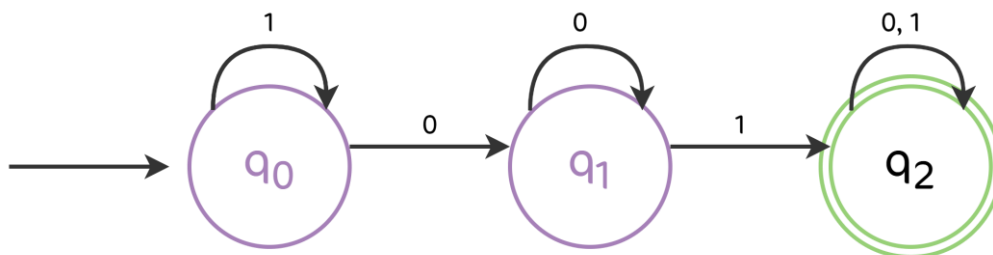
$$\Sigma^* 01 \Sigma^*$$

الـ Regular Expression هو:

إذاً وجود الصفر والواحد سيحقق لنا Accept State

ما هي خطوات بناء الـ Automaton؟1. عدد الحالات: $Q = \{ q_0, q_1, q_2 \}$ الحالة الابتدائية q_0 : ليس لها علاقة ومن المستحيل أن تكون Accept State، يجب علينا دراسة الحالة الوسطية q_1

$$\begin{aligned} q_0 - 0 &\rightarrow q_1 \\ q_1 - 1 &\rightarrow q_2 \end{aligned}$$

وبعدها مهما حصلت q_2 على دخل فهي Accept State

نذكر أن الحل ليس فريداً، أي يمكن أن نوجد حل آخر للمسألة، يمكن أن نحل بعدد حالات أكثر، يمكن أن نحل بعدد حالات أقل. سنتعلم لاحقاً كيف وفقاً للـ Rules فنقوم بـ Minimization

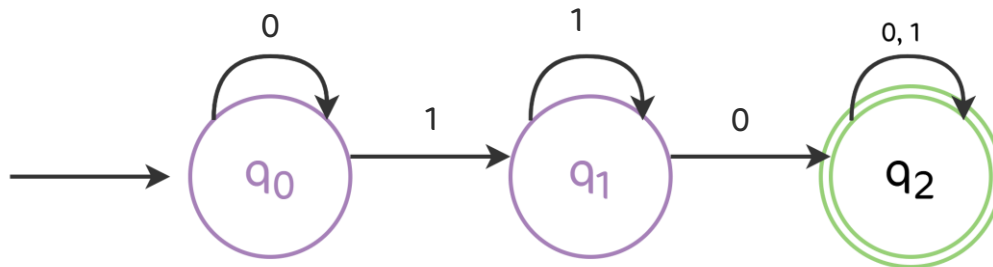
- $Q = \{ q_0, q_1, q_2 \}$
- $\Sigma = \{ 0, 1 \}$
- $Start\ State = q_0$
- $F = \{ q_2 \}$

$\delta = \text{Transition Table}$ 

$\delta =$	0	1
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_2	q_2

EX:

Build a DFA for the following language:

 $L = w \mid w \text{ is a binary string that contains } 10 \text{ as a substring}$ 

نتوقف عند الـ q_2 وذلك لوجود الـ "10"، فلن نقوم بالبحث عن البداية من جديد، لذلك السهام من q_2 كلها تلتف على نفسها (Loop) وتبقى في q_2

مثلاً لو دخلت لهذه الحالة السلسلة 1110:

$q_0 - 1 \rightarrow q_1$
 $q_1 - 1 \rightarrow q_1$
 $q_0 - 1 \rightarrow q_1$
 $q_1 - 0 \rightarrow q_2$

✓ تم العثور على "01"

1. What makes this DFA deterministic?

Deterministic كل حالة لها حالة واحدة فقط عند الانتقال عند Symbol واحد وهذا ما يجعلها

2. What if the language allows empty string (ϵ إبسيلون)?

سندرسها لاحقاً...

"Some Quote!"



Language of DFA

DFA يقبل String إذا وجد له مسار من q_0 إلى Accepting State عن طريق w ، أي بالنهاية w تتألف من أكثر من Symbol.

أي:

$L(A) = "L" \text{ is the language of Automaton "A"}$

أو:

Automaton "A" recognizes language "L"

$$L(A) = \{ w \mid \delta q_0, w \in F \}$$

هذه الحالة تتألف من w ، حيث w هي مجموعة الكلمات التي يكون فيها: إذا دخلت هذه الكلمة على Automaton انطلاقاً من q_0 فالخرج سيكون State تنتمي إلى Final Set of States F أو Accepting State، لأنه كما ذكرنا سابقاً أن F من الممكن أن تكون Accepting State واحدة أو أكثر من Accepting State، وبالتالي:

إذا أدى دخول هذه w إلى Automaton عن طريق تابع الانتقال والوصول إلى إحدى الحالات النهائية فهذا يعني أن w is accepted by the automaton A

$L(A) = \text{all strings that lead to an accepting state from } q_0$

Exercise 2:

Build a DFA for the following language:

$\{ w \mid w \text{ is a binary string that has even number of 1s and even number of 0s} \}$

الحل:

1) Define the DFA components:

❖ States: $Q = \{ q_{ee}, q_{e0}, q_{0e}, q_{00} \}$, where:

- q_{ee} : Even 0s, even 1s (start and accept state)
- q_{e0} : Even 0s, odd 1s
- q_{0e} : odd 0s, even 1s
- q_{00} : odd 0s, odd 1s

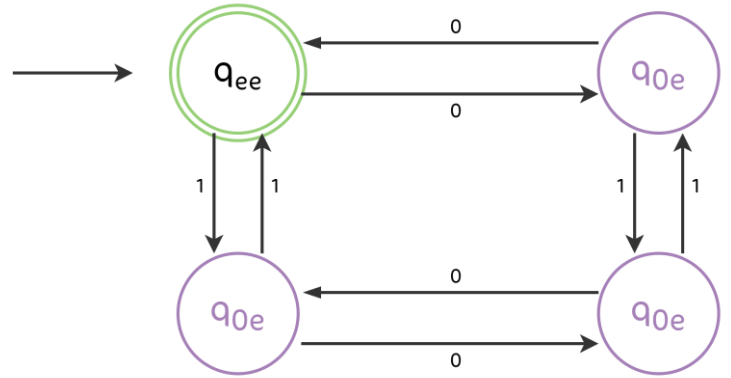
❖ Alphabet: $\Sigma = \{ 0, 1 \}$



❖ Start State: q_{ee} ❖ Accept State: q_{ee} 2) Transition Function δ :

The transition table is defined as follows:

State	Input 1	Input 2
q_{ee}	q_{0e}	q_{e0}
q_{e0}	q_{00}	q_{00}
q_{00}	q_{ee}	q_{0e}



Extension of Transitions (δ) to Paths ($\hat{\delta}$)

$\hat{\delta}(q, w) = \text{destination state from state } q \text{ on input } w$

مسار للحالة من q_0 إذا أخذ w input وطبعاً حسب w مما يتألف، أي إذا كان يتألف من "aab" نحتاج لدراسة a إلى أين ستذهب بال q وبعدها للحالة التي تم الانتقال لها ال b إلى أين ستذهب بال q

$$\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$$

هذا مسار ال DFA، ال Path الذي سلكناه: $\hat{\delta}(q, wa)$

انطلاقاً من ال q state عبر الكلمة w وبعدها ال "a" Symbol يساوي تماماً إلى $\delta(\hat{\delta}(q, w), a)$



Non-Deterministic Finite Automata (NFA)

Non-Deterministic: غير حتمي / غير محدد

تأتي فكرة عدم الحتمية من أجل أي حالة q_i إذا جاء Symbol معين قد يسبب ذلك الانتقال لأكثر من حالة، أي مثلاً:

أخذ 1 $\leftarrow$ ينتقل إلى q_j

أخذ 1 $\leftarrow$ ينتقل إلى q_k

q_i

■ نفس ال Symbol أخذنا إلى أكثر من حالة

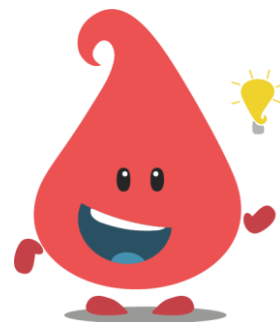
■ التصميم بال NFA أسهل منه في ال DFA

بفرض لدينا طريق يؤدي إلى Accepting State وطريق لـ Non-Accepting State،
في هذه الحالة أي طريق ستسلك الآلة؟

عملياً لدينا عدة مسارات (Different Paths) إذا أدى واحد من هذه المسارات على الأقل إلى Accepting State، هذا يعني أن الـ NFA تقبل هذا الـ w أو هذا الـ String، أي يجب أن تمر على كل المسارات والانتباه إذا كان واحد منها على الأقل يؤدي إلى Accepting State، وهذه هي فكرة الـ NFA

Formal Definition of NFA

- عدد الحالات States: Q
- أبجدية اللغة: Σ
- q_0 : Start State
- F : Set of Final / Accepting States
- تابع الانتقال: δ



الفرق الوحيد بين NFA و DFA هو تابع الانتقال δ

NFA	DFA
Symbol واحد بأكثر من حالة	من أجل كل Input Symbol حالة واحدة فقط

- δ : a transition function which is a mapping between $Q \times \Sigma \Rightarrow$ **subset of Q**
(وليس Q واحدة)
- An NFA is also defined by the 5-tuple $\{ Q, \Sigma, q_0, F, \delta \}$

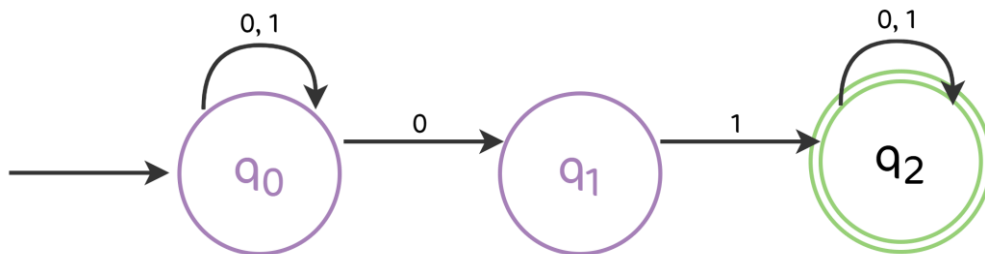
حل مثال DFA السابق ولكن بطريقة الـ NFA:

NFA for strings containing 01

$L = \{ w \mid w \text{ is a binary string that contains } 01 \text{ as a substring} \}$

إذا أوصلنا واحد من المسارات على الأقل للـ Final State ← w is accepted otherwise rejected

سيكون لدينا أكثر من مسار، سنرفض مساراً وستقبل مساراً، أو سنرفض أكثر من مسار ونقبل مساراً واحداً، أو سنقبل عدة مسارات إذا أدى مسار واحد للـ Final State أو لـ One of Final States



أول اختلاف عن الـ DFA:

- q_0 تسلك طريقين
- q_1 ليس لها طريق عندما يأتي إليها الصفر

انتبه: حيث أن في الـ DFA يجب أن يكون كل حالة لها انتقال محدد / معرف لكل رمز من رموز أبجدية اللغة

- $Q = \{ q_0, q_1, q_2 \}$
- $\Sigma = \{ 0, 1 \}$
- Start State = q_0
- $F = \{ q_2 \}$

Transition Table Symbols

δ	0	1
q_0	$\{ q_0, q_1 \}$	q_0
q_1	ϕ	q_2
q_2	q_2	q_2

هذا هو المسار الذي أوصلي إلى Accept State ← w مقبولة بال NFA

ماذا تعني ϕ ؟

تعني الكلمة وصلها 0 بال q_1 ، هذا مماثل تماماً لـ Dead State أو الحالة الميتة

هل الـ Dead State موجودة بال DFA؟

نعم موجودة (سنتعلمها بعد قليل)

هل عدم وجود Dead State بال NFA مشروع ومسموح به؟

نعم

هل وجود الحالة الميتة Dead State بال NFA يعني "غير الحتمية"؟

لا، لا يعني ذلك Non-Deterministic، وإنما يعني أن الـ NFA بتصميمها تسمح بعدم وجود ظاهري لـ Dead State أو لا Error State، أو بمرجعيات أخرى تسمى بالـ Trap State (حالة الفخ)، ببساطة من الممكن التعبير عن المجموعة الخالية ϕ بال NFA، ولا يجب الخلط بين فكرة Dead State وفكرة الـ Non-Deterministic.

نكمل بجل التمرين، سنختبر:

قد تسلك ال 0 وسينقلها لا q_0

q_0 على

وقد تسلك ال 0 وسينقلها لا q_1

التوازي

NFA تسلك عدة طرق بعضها يؤدي إلى Final State وبعضها لا يؤدي، ولكن إذا وجد على الأقل واحد يأخذني إلى ال Accepting State فال w مقبولة ضمن هذا ال NFA

مثال على الرفض:

لنأخذ "1110" يجب أن يكونوا متتاليين (Consecutive)

أخذ 1 ← ينتقل إلى q_0 (مسار واحد)

q_0

أخذ 1 ← ينتقل إلى q_0 أو q_1

ولا واحدة منهن من ال Final States وبالتالي تم رفض ال w بالحالتين ($w = \text{word}$)

لماذا NFA هي Non-Deterministic؟

لأنه ببساطة الحالة الواحدة ممكن أن تؤدي لأكثر من حالة

- لو أخذت ال q_1 الصفر فستؤدي إلى ϕ (مرفوضة أيضاً)

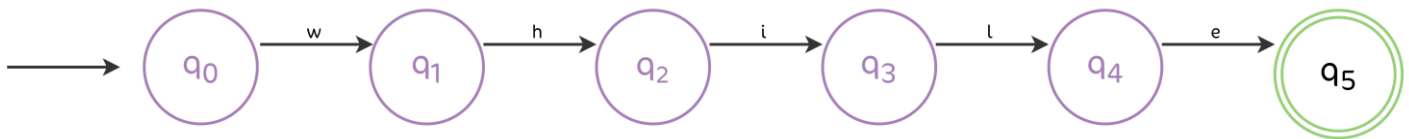
- الحالة الابتدائية أو أي حالة **يجب** أن يكون لها Symbol حتى يكون لها انتقال، لكن بال NFA ليس من الضروري تمثيل كل الحالات، مثلاً ال q_1 يمثلها ال $\text{input} = 1$ ولكن لا يمثلها ال $\text{input} = 0$ ، **ضمنياً** الصفر يؤدي إلى مجموعة هي ϕ

- هذا التمثيل الضمني مسموح به بال NFA وغير مسموح به بال DFA

مثال على التمثيل الضمني:

- لغة تتألف من كلمة واحدة هي while، وبالتالي أريد تصميم DFA، NFA لا يقبلون إلا الكلمة while
- لو كانت الكلمة whele ترفض، while ترفض، وهذا تطبيق مباشر جداً على تحليل المعجمية Lexical Analysis

الـ NFA:



q_0 إذا حصلت على h تذهب إلى ϕ ضمناً، لكن لا يجوز هذا بالـ DFA.

بالـ DFA الوضع مختلف، لا يمكننا القيام بإعادة تفسير ضمني بل يجب أن يكون لدينا State اسمها Error, Trap, Dead

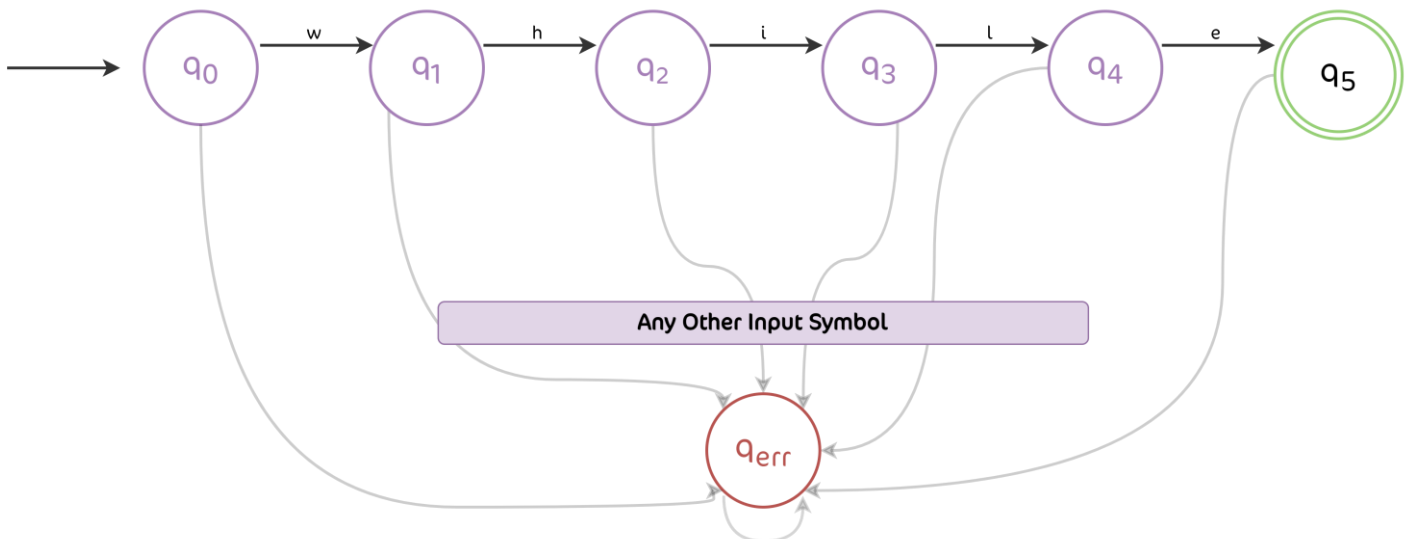
What is an “Error State” or “Dead State” or “Trap State”

هي الحالة التي نقوم بعمل رفضي للأوتوماتا عند الوصول إليها

- NFA أسرع من الـ DFA بالتصميم

- تصميم الـ NFA أسهل من تصميم الـ DFA

A DFA for recognizing the keyword “While”:



But transitions into a dead state in NFA are implicit (ضمني)